webon汎用照会 値のベリファイ用ユーティリティ

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-100
画面名	webon汎用照会 値のベリファイ用ユーティリティ
会社名	
作成日	

概要

概要説明

webon汎用照会 値のベリファイ用ユーティリティの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	N2_SIGHT.Ｂ顧客基本表、受給有無テーブル、年金1_LK、年金種類テーブル、年金__削除明細、店名テーブル、性別テーブル、裁定請求テーブル

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

doubleCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「doubleCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理2：詳細仕様参照）。

1.2. 通常処理：成立の結果を返す。この経路の後続手順は実行しない。

1-例外1. 例外条件「NumberFormatException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：不成立の結果を返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	成立の結果を返す
条件3を満たす（原本条件は補足説明を参照）	不成立の結果を返す

intCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「intCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理2：詳細仕様参照）。

1.2. 通常処理：成立の結果を返す。この経路の後続手順は実行しない。

1-例外1. 例外条件「NumberFormatException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：不成立の結果を返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	成立の結果を返す
条件3を満たす（原本条件は補足説明を参照）	不成立の結果を返す

numCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「numCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

numCheck2

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「numCheck2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

amountCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「amountCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

amountCheck2

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「amountCheck2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

alphaCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「alphaCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

alphaCheck2

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「alphaCheck2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

digitMaxCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「digitMaxCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

digitMinCheck

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「digitMinCheck」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

numberForm

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「numberForm」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 「条件9を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：「条件10を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.1.1. 条件成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

3.1-別.1. 条件不成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

3.2. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件13を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
条件14を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

EnToEm

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「EnToEm」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。
2. 繰返し条件「条件9を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

2.2. 繰返しの各回：「条件11を満たす（原本条件は補足説明を参照）、または条件12を満たす（原本条件は補足説明を参照）、または条件13を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.2.1. 条件成立時：「対象項目」を設定する。

1. 関連情報を取得・設定する（処理14：詳細仕様参照）。

2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

EmToEn

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「EmToEn」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。
2. 繰返し条件「条件9を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

2.2. 繰返しの各回：「条件11を満たす（原本条件は補足説明を参照）、または条件12を満たす（原本条件は補足説明を参照）、または条件13を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.2.1. 条件成立時：「対象項目」を設定する。

1. 関連情報を取得・設定する（処理14：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

ZenToHanForKKana

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「ZenToHanForKKana」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 繰返し条件「条件11を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

2.2. 繰返しの各回：繰返し条件「条件13を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.2.1. 繰返しの各回：「条件14を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.2.1.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

2.2.1.2. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

2.3. 繰返しの各回：「条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.3.1. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkZenToHanKana

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「checkZenToHanKana」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 繰返し条件「条件11を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

2.2. 繰返しの各回：繰返し条件「条件13を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.2.1. 繰返しの各回：「条件14を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.2.1.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

2.2.1.2. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

2.3. 繰返しの各回：「条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.3.1. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkZenCodeBool

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「checkZenCodeBool」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または条件9を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：不成立の結果を返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 指定した条件でデータを照会する。
3. 繰返し条件「条件11を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

4.2. 繰返しの各回：関連情報を取得・設定する（処理13：詳細仕様参照）。

4.3. 繰返しの各回：関連情報を取得・設定する（処理14：詳細仕様参照）。

4.4. 繰返しの各回：「条件15を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.4.1. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない、または条件17を満たす（原本条件は補足説明を参照）	不成立の結果を返す
NOT (val が等しい 未設定||val.equals(""))	処理結果を呼出元へ返す

checkZenCodeStr

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「checkZenCodeStr」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または条件15を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理16：詳細仕様参照）。
2. 関連情報を取得・設定する（処理17：詳細仕様参照）。
3. 指定した条件でデータを照会する。
4. 繰返し条件「条件18を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2. 繰返しの各回：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.3. 繰返しの各回：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.4. 繰返しの各回：「条件22を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.4.1. 条件成立時：関連情報を取得・設定する（処理23：詳細仕様参照）。

5.4.2. 条件成立時：関連情報を取得・設定する（処理24：詳細仕様参照）。

1. 関連情報を取得・設定する（処理25：詳細仕様参照）。
2. 繰返し条件「条件26を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

7.1. 繰返しの各回：関連情報を取得・設定する（処理27：詳細仕様参照）。

1. 関連情報を取得・設定する（処理28：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない、または条件29を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (val が等しい 未設定||val.equals(""))	処理結果を呼出元へ返す

checkShinkinCodeStr

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「checkShinkinCodeStr」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または条件27を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理28：詳細仕様参照）。
2. 関連情報を取得・設定する（処理29：詳細仕様参照）。
3. 繰返し条件「条件30を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理31：詳細仕様参照）。

1. 関連情報を取得・設定する（処理32：詳細仕様参照）。
2. 関連情報を取得・設定する（処理33：詳細仕様参照）。
3. 関連情報を取得・設定する（処理34：詳細仕様参照）。
4. 関連情報を取得・設定する（処理35：詳細仕様参照）。
5. 関連情報を取得・設定する（処理36：詳細仕様参照）。
6. 関連情報を取得・設定する（処理37：詳細仕様参照）。
7. 繰返し条件「条件38を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

11.1. 繰返しの各回：関連情報を取得・設定する（処理39：詳細仕様参照）。

11.2. 繰返しの各回：関連情報を取得・設定する（処理40：詳細仕様参照）。

11.3. 繰返しの各回：関連情報を取得・設定する（処理41：詳細仕様参照）。

11.4. 繰返しの各回：指定した条件でデータを照会する。

11.5. 繰返しの各回：「条件42を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

11.5.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

11.5-別.1. 条件不成立時：「bytArr.length が等しい 2」に該当するか確認する。成立時と不成立時の手順を分けて行う。

11.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理43：詳細仕様参照）。

11.5-別.1.2. 条件成立時：関連情報を取得・設定する（処理44：詳細仕様参照）。

11.5-別.1.3. 条件成立時：関連情報を取得・設定する（処理45：詳細仕様参照）。

11.5-別.1.4. 条件成立時：「条件46を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.5-別.1.4.1. 条件成立時：関連情報を取得・設定する（処理47：詳細仕様参照）。

11.5-別.1-別.1. 条件不成立時：「bytArr.length が等しい 1」に該当するか確認する。成立時と不成立時の手順を分けて行う。

11.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理48：詳細仕様参照）。

11.5-別.1-別.1-別.1. 条件不成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理49：詳細仕様参照）。
2. 繰返し条件「条件50を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

13.1. 繰返しの各回：関連情報を取得・設定する（処理51：詳細仕様参照）。

1. 関連情報を取得・設定する（処理52：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない、または条件53を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (val が等しい 未設定||val.equals("")) ・条件54を満たす（原本条件は補足説明を参照） ・条件55を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

到達条件	結果
NOT (val が等しい 未設定||val.equals("")) ・条件56を満たす（原本条件は補足説明を参照） ・条件57を満たす（原本条件は補足説明を参照） ・NOT (bytArr.length が等しい 2) ・NOT (bytArr.length が等しい 1)	処理の異常を通知し、この経路を終了する
NOT (val が等しい 未設定||val.equals(""))	処理結果を呼出元へ返す

convertUniTag2Esc

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「convertUniTag2Esc」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理29：詳細仕様参照）。
2. 関連情報を取得・設定する（処理30：詳細仕様参照）。
3. 関連情報を取得・設定する（処理31：詳細仕様参照）。
4. 関連情報を取得・設定する（処理32：詳細仕様参照）。
5. 関連情報を取得・設定する（処理33：詳細仕様参照）。
6. 関連情報を取得・設定する（処理34：詳細仕様参照）。
7. 関連情報を取得・設定する（処理35：詳細仕様参照）。
8. 関連情報を取得・設定する（処理36：詳細仕様参照）。
9. 関連情報を取得・設定する（処理37：詳細仕様参照）。
10. 繰返し条件「条件38を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

10.1. 繰返しの各回：関連情報を取得・設定する（処理39：詳細仕様参照）。

10.2. 繰返しの各回：関連情報を取得・設定する（処理40：詳細仕様参照）。

10.3. 繰返しの各回：対象データを削除する。

10.4. 繰返しの各回：「対象項目」を設定する。

10.5. 繰返しの各回：関連情報を取得・設定する（処理41：詳細仕様参照）。

10.6. 繰返しの各回：関連情報を取得・設定する（処理42：詳細仕様参照）。

10.7. 繰返しの各回：対象データを削除する。

10.8. 繰返しの各回：関連情報を取得・設定する（処理43：詳細仕様参照）。

10.9. 繰返しの各回：「条件44を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

10.9.1. 条件成立時：関連情報を取得・設定する（処理45：詳細仕様参照）。

10.9.2. 条件成立時：「条件46を満たす（原本条件は補足説明を参照）、かつ条件47を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

10.9.2.1. 条件成立時：関連情報を取得・設定する（処理48：詳細仕様参照）。

10.9.2.2. 条件成立時：関連情報を取得・設定する（処理49：詳細仕様参照）。

10.9.2.3. 条件成立時：関連情報を取得・設定する（処理50：詳細仕様参照）。

10.9.2-別.1. 条件不成立時：関連情報を取得・設定する（処理51：詳細仕様参照）。

10.9-別.1. 条件不成立時：関連情報を取得・設定する（処理52：詳細仕様参照）。

10.10. 繰返しの各回：関連情報を取得・設定する（処理53：詳細仕様参照）。

10.11. 繰返しの各回：「条件54を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

10.11.1. 条件成立時：関連情報を取得・設定する（処理55：詳細仕様参照）。

1. 関連情報を取得・設定する（処理56：詳細仕様参照）。

2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照

情報	内容
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkGaijiCodeStr

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「checkGaijiCodeStr」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または条件6を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：不成立の結果を返す。この経路の後続手順は実行しない。

1. 指定した条件でデータを照会する。
2. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理8：詳細仕様参照）。

3.2. 繰返しの各回：関連情報を取得・設定する（処理9：詳細仕様参照）。

3.3. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

3.4. 繰返しの各回：「入力値の制約と変換結果が成立する」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.4.1. 条件成立時：成立の結果を返す。この経路の後続手順は実行しない。

1. 不成立の結果を返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない、または条件11を満たす（原本条件は補足説明を参照）	不成立の結果を返す
NOT (val が等しい 未設定||val.equals(""))・条件12を満たす（原本条件は補足説明を参照）・入力値の制約と変換結果が成立する	成立の結果を返す
NOT (val が等しい 未設定||val.equals(""))	不成立の結果を返す

checkRange

処理概要

webon汎用照会 値のベリファイ用ユーティリティで「checkRange」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 「条件6を満たす（原本条件は補足説明を参照）、かつ条件7を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

DB関連処理

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
doubleCheck	処理1	Double.parseDouble(str)
doubleCheck	処理2	Double.parseDouble(str)
doubleCheck	条件3	NumberFormatException e
intCheck	処理1	Integer.parseInt(str)
intCheck	処理2	Integer.parseInt(str)

イベント	参照番号	原本条件・処理
intCheck	条件3	NumberFormatException e
numCheck	処理1	final String P_ALPHA_DIGIT_ONLY = "^[0-9]+$"
numCheck	処理2	final String P_ALPHA_DIGIT_ONLY = "^[0-9]+$"
numCheck2	処理1	str=EmToEn(str)
numCheck2	処理2	str=EmToEn(str)
amountCheck	処理1	final String P_ALPHA_DIGIT_ONLY = "^[0-9,]+$"
amountCheck	処理2	final String P_ALPHA_DIGIT_ONLY = "^[0-9,]+$"
amountCheck2	処理1	final String P_ALPHA_DIGIT_ONLY = "^[0-9,-]+$"
amountCheck2	処理2	final String P_ALPHA_DIGIT_ONLY = "^[0-9,-]+$"
alphaCheck	処理1	final String P_ALPHA_DIGIT_ONLY = "^[0-9A-Za-z]+$"
alphaCheck	処理2	final String P_ALPHA_DIGIT_ONLY = "^[0-9A-Za-z]+$"
alphaCheck2	処理1	str=EmToEn(str)
alphaCheck2	処理2	str=EmToEn(str)
numberForm	処理1	result=str.substring(str.length()-dig)
numberForm	処理2	result=String.format("%1$0"+dig+"d", Integer.parseInt(str))
numberForm	条件3	str.length()>dig
numberForm	条件4	numCheck(str)
numberForm	処理5	str = EmToEn(str)
numberForm	処理6	String result=""

イベント	参照番号	原本条件・処理
numberForm	処理7	String result=""
numberForm	処理8	str = EmToEn(str)
numberForm	条件9	numCheck(str)
numberForm	条件10	str.length()>dig
numberForm	処理11	result=str.substring(str.length()-dig)
numberForm	処理12	result=String.format("%1$0"+dig+"d", Integer.parseInt(str))
numberForm	条件13	numCheck(str)
numberForm	条件14	NOT (numCheck(str))
EnToEm	処理1	value = sb.toString()
EnToEm	条件2	int i = 0; i < sb.length(); i++
EnToEm	条件3	(c >= 0x30 && c <= 0x39)
EnToEm	条件4	(c >= 0x41 && c <= 0x5A)
EnToEm	条件5	(c >= 0x61 && c <= 0x7A)
EnToEm	処理6	int c = (int) sb.charAt(i)
EnToEm	処理7	StringBuilder sb = new StringBuilder(value)
EnToEm	処理8	StringBuilder sb = new StringBuilder(value)
EnToEm	条件9	int i = 0; i < sb.length(); i++
EnToEm	処理10	int c = (int) sb.charAt(i)
EnToEm	条件11	(c >= 0x30 && c <= 0x39)

イベント	参照番号	原本条件・処理
EnToEm	条件12	(c >= 0x41 && c <= 0x5A)
EnToEm	条件13	(c >= 0x61 && c <= 0x7A)
EnToEm	処理14	value = sb.toString()
EmToEn	処理1	value = sb.toString()
EmToEn	条件2	int i = 0; i < sb.length(); i++
EmToEn	条件3	(c >= 0xFF10 && c <= 0xFF19)
EmToEn	条件4	(c >= 0xFF21 && c <= 0xFF3A)
EmToEn	条件5	(c >= 0xFF41 && c <= 0xFF5A)
EmToEn	処理6	int c = (int) sb.charAt(i)
EmToEn	処理7	StringBuilder sb = new StringBuilder(value)
EmToEn	処理8	StringBuilder sb = new StringBuilder(value)
EmToEn	条件9	int i = 0; i < sb.length(); i++
EmToEn	処理10	int c = (int) sb.charAt(i)
EmToEn	条件11	(c >= 0xFF10 && c <= 0xFF19)
EmToEn	条件12	(c >= 0xFF21 && c <= 0xFF3A)
EmToEn	条件13	(c >= 0xFF41 && c <= 0xFF5A)
EmToEn	処理14	value = sb.toString()
ZenToHanForKKana	条件1	int i = 0, j = 0; i < p.length(); i++
ZenToHanForKKana	処理2	sb.append(c)

イベント	参照番号	原本条件・処理
ZenToHanForKKana	条件3	j >= kanaHanZenTbl.length
ZenToHanForKKana	条件4	j = 0; j < kanaHanZenTbl.length; j++
ZenToHanForKKana	処理5	break
ZenToHanForKKana	処理6	sb.append(kanaHanZenTbl[j][0])
ZenToHanForKKana	条件7	p.substring(i).startsWith(kanaHanZenTbl[j][1])
ZenToHanForKKana	処理8	Character c = new Character(p.charAt(i))
ZenToHanForKKana	処理9	StringBuffer sb = new StringBuffer()
ZenToHanForKKana	処理10	StringBuffer sb = new StringBuffer()
ZenToHanForKKana	条件11	int i = 0, j = 0; i < p.length(); i++
ZenToHanForKKana	処理12	Character c = new Character(p.charAt(i))
ZenToHanForKKana	条件13	j = 0; j < kanaHanZenTbl.length; j++
ZenToHanForKKana	条件14	p.substring(i).startsWith(kanaHanZenTbl[j][1])
ZenToHanForKKana	処理15	sb.append(kanaHanZenTbl[j][0])
ZenToHanForKKana	処理16	break
ZenToHanForKKana	条件17	j >= kanaHanZenTbl.length
ZenToHanForKKana	処理18	sb.append(c)
checkZenToHanKana	条件1	int i = 0, j = 0; i < p.length(); i++
checkZenToHanKana	処理2	sb.append(c)
checkZenToHanKana	条件3	j >= kanaCheckTbl.length

イベント	参照番号	原本条件・処理
checkZenToHanKana	条件4	j = 0; j < kanaCheckTbl.length; j++
checkZenToHanKana	処理5	break
checkZenToHanKana	処理6	sb.append(kanaCheckTbl[j][0])
checkZenToHanKana	条件7	p.substring(i).startsWith(kanaCheckTbl[j][1])
checkZenToHanKana	処理8	Character c = new Character(p.charAt(i))
checkZenToHanKana	処理9	StringBuffer sb = new StringBuffer()
checkZenToHanKana	処理10	StringBuffer sb = new StringBuffer()
checkZenToHanKana	条件11	int i = 0, j = 0; i < p.length(); i++
checkZenToHanKana	処理12	Character c = new Character(p.charAt(i))
checkZenToHanKana	条件13	j = 0; j < kanaCheckTbl.length; j++
checkZenToHanKana	条件14	p.substring(i).startsWith(kanaCheckTbl[j][1])
checkZenToHanKana	処理15	sb.append(kanaCheckTbl[j][0])
checkZenToHanKana	処理16	break
checkZenToHanKana	条件17	j >= kanaCheckTbl.length
checkZenToHanKana	処理18	sb.append(c)
checkZenCodeBool	条件1	int i=0;i+1<br.length;i+=2
checkZenCodeBool	処理2	flg=true
checkZenCodeBool	条件3	!(checkRange(sj, "8140", "817A")|| checkRange(sj, "824F", "8396")|| checkRange(sj, "889F", "9872")|| checkRange(sj, "989F", "EAA4"))

イベント	参照番号	原本条件・処理
checkZenCodeBool	処理4	String sj = Integer.toHexString(b1 & 0xff)+Integer.toHexString(b2 & 0xff)
checkZenCodeBool	処理5	byte b2=br[i+1]
checkZenCodeBool	処理6	byte b1=br[i]
checkZenCodeBool	処理7	boolean flg=false
checkZenCodeBool	条件8	val.equals("")
checkZenCodeBool	条件9	val.equals("")
checkZenCodeBool	処理10	boolean flg=false
checkZenCodeBool	条件11	int i=0;i+1<br.length;i+=2
checkZenCodeBool	処理12	byte b1=br[i]
checkZenCodeBool	処理13	byte b2=br[i+1]
checkZenCodeBool	処理14	String sj = Integer.toHexString(b1 & 0xff)+Integer.toHexString(b2 & 0xff)
checkZenCodeBool	条件15	!(checkRange(sj, "8140", "817A")|| checkRange(sj, "824F", "8396")|| checkRange(sj, "889F", "9872")|| checkRange(sj, "989F", "EAA4"))
checkZenCodeBool	処理16	flg=true
checkZenCodeBool	条件17	val.equals("")
checkZenCodeStr	処理1	ngStr=new String(ngBr)
checkZenCodeStr	条件2	int i=0;i<ngChrList.size();i++
checkZenCodeStr	処理3	ngBr[i]=ngChrList.get(i)
checkZenCodeStr	処理4	byte ngBr[] = new byte[ngChrList.size()]

イベント	参照番号	原本条件・処理
checkZenCodeStr	条件5	int i=0;i+1<br.length;i+=2
checkZenCodeStr	処理6	ngChrList.add(b2)
checkZenCodeStr	処理7	ngChrList.add(b1)
checkZenCodeStr	条件8	!(checkRange(sj, "8140", "817A")|| checkRange(sj, "824F", "8396")|| checkRange(sj, "889F", "9872")|| checkRange(sj, "989F", "EAA4"))
checkZenCodeStr	処理9	String sj = Integer.toHexString(b1 & 0xff)+Integer.toHexString(b2 & 0xff)
checkZenCodeStr	処理10	byte b2=br[i+1]
checkZenCodeStr	処理11	byte b1=br[i]
checkZenCodeStr	処理12	ArrayList<Byte> ngChrList = new ArrayList<Byte>()
checkZenCodeStr	処理13	String ngStr=""
checkZenCodeStr	条件14	val.equals("")
checkZenCodeStr	条件15	val.equals("")
checkZenCodeStr	処理16	String ngStr=""
checkZenCodeStr	処理17	ArrayList<Byte> ngChrList = new ArrayList<Byte>()
checkZenCodeStr	条件18	int i=0;i+1<br.length;i+=2
checkZenCodeStr	処理19	byte b1=br[i]
checkZenCodeStr	処理20	byte b2=br[i+1]
checkZenCodeStr	処理21	String sj = Integer.toHexString(b1 & 0xff)+Integer.toHexString(b2 & 0xff)

イベント	参照番号	原本条件・処理
checkZenCodeStr	条件22	!(checkRange(sj, "8140", "817A")|| checkRange(sj, "824F", "8396")|| checkRange(sj, "889F", "9872")|| checkRange(sj, "989F", "EAA4"))
checkZenCodeStr	処理23	ngChrList.add(b1)
checkZenCodeStr	処理24	ngChrList.add(b2)
checkZenCodeStr	処理25	byte ngBr[] = new byte[ngChrList.size()]
checkZenCodeStr	条件26	int i=0;i<ngChrList.size();i++
checkZenCodeStr	処理27	ngBr[i]=ngChrList.get(i)
checkZenCodeStr	処理28	ngStr=new String(ngBr)
checkZenCodeStr	条件29	val.equals("")
checkShinkinCodeStr	処理1	String ngStr=new String(charAr)
checkShinkinCodeStr	条件2	int i=0;i<ngChList.size();i++
checkShinkinCodeStr	処理3	charAr[i]=ngChList.get(i).charValue()
checkShinkinCodeStr	処理4	char[] charAr =new char[ngChList.size()]
checkShinkinCodeStr	条件5	it.hasNext()
checkShinkinCodeStr	処理6	ngChList.add(chkChr)
checkShinkinCodeStr	条件7	!(checkRange(chkCode, "8140", "EAA4")|| checkRange(chkCode, "F040", "F2C8"))
checkShinkinCodeStr	処理8	chkCode = Integer.toHexString(byt1 & 0xff) +Integer.toHexString(byt2 & 0xff)
checkShinkinCodeStr	処理9	byt2=bytArr[1]
checkShinkinCodeStr	処理10	byt1=bytArr[0]

イベント	参照番号	原本条件・処理
checkShinkinCodeStr	処理11	ngChList.add(chkChr)
checkShinkinCodeStr	条件12	bytArr.length>2
checkShinkinCodeStr	処理13	byt2=0
checkShinkinCodeStr	処理14	byt1=0
checkShinkinCodeStr	処理15	chkChr=it.next()
checkShinkinCodeStr	処理16	String chkCode
checkShinkinCodeStr	処理17	byte byt1,byt2
checkShinkinCodeStr	処理18	byte[] bytArr
checkShinkinCodeStr	処理19	Character chkChr
checkShinkinCodeStr	処理20	Iterator<Character> it = chList.iterator()
checkShinkinCodeStr	処理21	List<Character> ngChList = new ArrayList<Character>()
checkShinkinCodeStr	条件22	Character ch:charr
checkShinkinCodeStr	処理23	chList.add(ch)
checkShinkinCodeStr	処理24	List<Character> chList = new ArrayList<Character>()
checkShinkinCodeStr	処理25	char[] charr= val.toCharArray()
checkShinkinCodeStr	条件26	val.equals("")
checkShinkinCodeStr	条件27	val.equals("")
checkShinkinCodeStr	処理28	char[] charr= val.toCharArray()
checkShinkinCodeStr	処理29	List<Character> chList = new ArrayList<Character>()

イベント	参照番号	原本条件・処理
checkShinkinCodeStr	条件30	Character ch:charr
checkShinkinCodeStr	処理31	chList.add(ch)
checkShinkinCodeStr	処理32	List<Character> ngChList = new ArrayList<Character>()
checkShinkinCodeStr	処理33	Iterator<Character> it = chList.iterator()
checkShinkinCodeStr	処理34	Character chkChr
checkShinkinCodeStr	処理35	byte[] bytArr
checkShinkinCodeStr	処理36	byte byt1,byt2
checkShinkinCodeStr	処理37	String chkCode
checkShinkinCodeStr	条件38	it.hasNext()
checkShinkinCodeStr	処理39	chkChr=it.next()
checkShinkinCodeStr	処理40	byt1=0
checkShinkinCodeStr	処理41	byt2=0
checkShinkinCodeStr	条件42	bytArr.length>2
checkShinkinCodeStr	処理43	byt1=bytArr[0]
checkShinkinCodeStr	処理44	byt2=bytArr[1]
checkShinkinCodeStr	処理45	chkCode = Integer.toHexString(byt1 & 0xff) +Integer.toHexString(byt2 & 0xff)
checkShinkinCodeStr	条件46	!(checkRange(chkCode, "8140", "EAA4")|| checkRange(chkCode, "F040", "F2C8"))
checkShinkinCodeStr	処理47	ngChList.add(chkChr)

イベント	参照番号	原本条件・処理
checkShinkinCodeStr	処理48	ngChList.add(chkChr)
checkShinkinCodeStr	処理49	char[] charAr =new char[ngChList.size()]
checkShinkinCodeStr	条件50	int i=0;i<ngChList.size();i++
checkShinkinCodeStr	処理51	charAr[i]=ngChList.get(i).charValue()
checkShinkinCodeStr	処理52	String ngStr=new String(charAr)
checkShinkinCodeStr	条件53	val.equals("")
checkShinkinCodeStr	条件54	it.hasNext()
checkShinkinCodeStr	条件55	bytArr.length>2
checkShinkinCodeStr	条件56	it.hasNext()
checkShinkinCodeStr	条件57	NOT (bytArr.length>2)
convertUniTag2Esc	処理1	sb2.append(sb1)
convertUniTag2Esc	条件2	sb1.indexOf(shead)>=0
convertUniTag2Esc	処理3	break
convertUniTag2Esc	条件4	count>100
convertUniTag2Esc	処理5	count++
convertUniTag2Esc	処理6	sb2.append(thead).append(Integer.toHexString(uppeer)).append(thead).append(Integer.toHexString(lower))
convertUniTag2Esc	処理7	int lower=(unicode-0x10000)%0x400+0xDC00
convertUniTag2Esc	処理8	int uppeer=(unicode-0x10000)/0x400+0xD800
convertUniTag2Esc	処理9	sb2.append(thead).append(Integer.toHexString(unicode))

イベント	参照番号	原本条件・処理
convertUniTag2Esc	条件10	unicode>=0x10000
convertUniTag2Esc	条件11	unicode <=0x10FFFF
convertUniTag2Esc	処理12	unicode = Integer.parseInt(uniStr)
convertUniTag2Esc	処理13	sb2.append(shead).append(uniStr).append(sfoot)
convertUniTag2Esc	条件14	intCheck(uniStr)
convertUniTag2Esc	処理15	uniStr=sbTmp.toString()
convertUniTag2Esc	処理16	sbTmp.append(sb1.subSequence(shead.length(), en))
convertUniTag2Esc	処理17	int en = sb1.indexOf(sfoot)
convertUniTag2Esc	処理18	sb2.append(sb1.subSequence(0, st))
convertUniTag2Esc	処理19	int st = sb1.indexOf(shead)
convertUniTag2Esc	処理20	int count=0
convertUniTag2Esc	処理21	String uniStr=""
convertUniTag2Esc	処理22	int unicode=0
convertUniTag2Esc	処理23	final String thead="\u"
convertUniTag2Esc	処理24	final String sfoot=";"
convertUniTag2Esc	処理25	final String shead="&#"
convertUniTag2Esc	処理26	StringBuilder sbTmp = new StringBuilder()
convertUniTag2Esc	処理27	StringBuilder sb2 = new StringBuilder()
convertUniTag2Esc	処理28	StringBuilder sb1 = new StringBuilder(ngString)

イベント	参照番号	原本条件・処理
convertUniTag2Esc	処理29	StringBuilder sb1 = new StringBuilder(ngString)
convertUniTag2Esc	処理30	StringBuilder sb2 = new StringBuilder()
convertUniTag2Esc	処理31	StringBuilder sbTmp = new StringBuilder()
convertUniTag2Esc	処理32	final String shead="&#"
convertUniTag2Esc	処理33	final String sfoot=";"
convertUniTag2Esc	処理34	final String thead="\u"
convertUniTag2Esc	処理35	int unicode=0
convertUniTag2Esc	処理36	String uniStr=""
convertUniTag2Esc	処理37	int count=0
convertUniTag2Esc	条件38	sb1.indexOf(shead)>=0
convertUniTag2Esc	処理39	int st = sb1.indexOf(shead)
convertUniTag2Esc	処理40	sb2.append(sb1.subSequence(0, st))
convertUniTag2Esc	処理41	int en = sb1.indexOf(sfoot)
convertUniTag2Esc	処理42	sbTmp.append(sb1.subSequence(shead.length(), en))
convertUniTag2Esc	処理43	uniStr=sbTmp.toString()
convertUniTag2Esc	条件44	intCheck(uniStr)
convertUniTag2Esc	処理45	unicode = Integer.parseInt(uniStr)
convertUniTag2Esc	条件46	unicode>=0x10000
convertUniTag2Esc	条件47	unicode <=0x10FFFF

イベント	参照番号	原本条件・処理
convertUniTag2Esc	処理48	int uppeer=(unicode-0x10000)/0x400+0xD800
convertUniTag2Esc	処理49	int lower=(unicode-0x10000)%0x400+0xDC00
convertUniTag2Esc	処理50	sb2.append(thead).append(Integer.toHexString(uppeer)).append(thead).append(Integer.toHexString(lower))
convertUniTag2Esc	処理51	sb2.append(thead).append(Integer.toHexString(unicode))
convertUniTag2Esc	処理52	sb2.append(shead).append(uniStr).append(sfoot)
convertUniTag2Esc	処理53	count++
convertUniTag2Esc	条件54	count>100
convertUniTag2Esc	処理55	break
convertUniTag2Esc	処理56	sb2.append(sb1)
checkGaijiCodeStr	条件1	int i=0;i+1<br.length;i+=2
checkGaijiCodeStr	処理2	String sj = Integer.toHexString(b1 & 0xff)+Integer.toHexString(b2 & 0xff)
checkGaijiCodeStr	処理3	byte b2=br[i+1]
checkGaijiCodeStr	処理4	byte b1=br[i]
checkGaijiCodeStr	条件5	val.equals("")
checkGaijiCodeStr	条件6	val.equals("")
checkGaijiCodeStr	条件7	int i=0;i+1<br.length;i+=2
checkGaijiCodeStr	処理8	byte b1=br[i]
checkGaijiCodeStr	処理9	byte b2=br[i+1]

イベント	参照番号	原本条件・処理
checkGaijiCodeStr	処理10	String sj = Integer.toHexString(b1 & 0xff)+Integer.toHexString(b2 & 0xff)
checkGaijiCodeStr	条件11	val.equals("")
checkGaijiCodeStr	条件12	int i=0;i+1<br.length;i+=2
checkRange	処理1	flg=true
checkRange	条件2	(Integer.parseInt(r1,16))<=Integer.parseInt(sj, 16)
checkRange	条件3	Integer.parseInt(sj,16)<=(Integer.parseInt(r2, 16))
checkRange	処理4	boolean flg=false
checkRange	処理5	boolean flg=false
checkRange	条件6	(Integer.parseInt(r1,16))<=Integer.parseInt(sj, 16)
checkRange	条件7	Integer.parseInt(sj,16)<=(Integer.parseInt(r2, 16))
checkRange	処理8	flg=true